THE UNIVERSITY OF SYDNEY

MTRX3760 - Git Tutorial

This is a getting started guide for using git. It covers the *mechanics* of using git. Please review the following to understand some of the *strategies*, *workflows, and best practices* for using revision control effectively including what should go into revision control and what should not, and how to make meaningful commits to a repo:
https://homes.cs.washington.edu/~mernst/advice/version-control.html

This tutorial is written for repos hosted on bitbucket. Similar mechanics exist for other hosting options, and github is another very good choice for hosting private individual or team repos. This tutorial is largely a shorter version of a more complete introduction like https://www.atlassian.com/git/tutorials/learn-git-with-bitbucket-cloud .

Plagiarism

Git makes it easy to share code. Please be mindful of what code you share with others, recalling that facilitating plagiarism of assignments is unacceptable behaviour that can have serious consequences.

Creating a Repo

If you don't already have one, create an account on bitbucket.org. Alternatives like github are also good, but we're going to use bitbucket here because of their extensive and well-written online tutorial content.

Create a private repo called `mtrx3760lab3`, following the instructions here: https://www.atlassian.com/git/tutorials/learn-git-with-bitbucket-cloud#create-the-repository stopping after completing "Step 2. Add a file to your local repository and put it on Bitbucket".

For the major project, you'll be working in a team. Create only one repository and grant team members access to the repo. Use bitbucket's `Settings` → `Users and Group Access` to allow access. All members of the group should clone the repo to different computers / folders.

When completing this tutorial consider working with a partner, or if you're working alone you'll need to play the role of two team members, and clone the repo to two different locations. Use the `git clone` command, specifying the destination folder as the final argument.

[UPDATED v2022.01] If you clone using HTTPS, you will by default need to enter your username and password for each operation involving the remote repo. You can avoid having to do this by following these instructions (from https://www.phys.uconn.edu/~rozman/Courses/P2200_15F/downloads/github-caching_password_in_git.pdf):

THE UNIVERSITY OF SYDNEY

In a terminal enter:
`git config --global credential.helper cache`
to set git to use the credential memory cache, then
`git config --global credential.helper 'cache --timeout=3600'`
To set the cache to timeout after 1 hour (setting is in seconds).

If you use github, then the github command line interface (CLI) offers an elegant approach to do the sam: https://help.github.com/en/articles/caching-your-github-password-in-git .

SSH cloning is more secure but more complex to set up. Follow these instructions to add an ssh key to your bitbucket account: https://confluence.atlassian.com/bitbucket/set-up-an-ssh-key-728138079.html .

At this point you should have your repo cloned to two locations, which we'll call **Clone A** and **Clone B**.

Add, commit, push

Similar to the procedure for creating a file, in **Clone A** do the following:

Create a folder called Q0.Git, and inside that folder create a file `hello.cpp`. Implement a simple hello world example in the file. Compile the example to `a.out`.

Do the following:
`git status`: check the status of the repo
`git add hello.cpp`: stage the new source file to be added, but not the binary
`git status`: check the status of the repo, shows that the file is staged but not committed
`git commit -m"adding hello.cpp"`: commit the new file to the local repo
`git status`: shows the file is committed locally but not pushed to repo
`git push`: push the local changes to the remote repo
`git status`: shows that your binary file hasn't been added to the repo

Diff

After doing the above, in **Clone B** do the following
*Do these steps without taking any action in **Clone A**, so that we can create an example of a merge conflict*
`git pull`: grab the changes made so far
Edit hello.cpp to also say hello in a second language
`git status`: shows you've made changes
`git diff`: shows what changes you've made
Add, commit and push your changes to the remote repo

Ignore Files

Create a `.gitignore` file in the top folder of your repo, containing the text `a.out`



`git status`: notice that this now ignores a.out, but shows that `.gitignore` is a new file

Add `.gitignore` to your local repo, commit it, and push the change to remote repo.

Sometimes `<merge conflict>` happens

Back in **Clone A**, before pulling any changes from the remote repo, edit `hello.cpp` to repeat "Hello, World" multiple times. This conflicts with the edits made via **Clone B** to say hello in a second language.

Use `git add` and `git commit` to commit changes to the local repo

`git push`: Push your changes to the remote repo. This should cause an error, because you have attempted to push changes to the remote repo before pulling remote changes

Dealing with a merge conflict

`git pull`: because `hello.cpp` has been changed in more than one repo, a merge is required. git will attempt this merge automatically.

If the merge is successful you will be asked for a commit message describing the merge. The default commit message "Merge branch..." is fine, accept it by pressing `ctrl-x`.

If the merge is NOT successful, you must follow the instructions provided by git. Open `hello.cpp` with an editor and edit the file to resolve the conflicts manually. Follow the instructions git gives you carefully during the merge process.

Alternatively, you may wish to use a merge tool like meld to help deal with merge conflicts, see below. Compile and test your merged `hello.cpp`. If there are errors, correct them and commit the corrected file locally before pushing your changes to the remote repo.

History and undo

You can undo commits. The easiest is to undo local changes:

`git restore <fname>`: resets local, unstaged changes to a file
(older git versions use `git checkout --`)

`git restore --staged <fname>`: unstages a staged change
(older git versions use `git reset HEAD <fname>`)

Note that “git status” gives you context-aware hints about which of these commands might help.

For changes that have been committed:

`git log`: shows the history of changes made to your repo, each has a unique identity ("hash")

`git log --pretty=oneline`: more compact version of the above

`git show <hash>` shows differences introduced in a past commit

`git revert <hash>` undoes the changes made in a past commit

There is additional information on undoing changes with git here:



https://www.atlassian.com/git/tutorials/learn-undoing-changes-with-bitbucket

A Simple Workflow

There's much more to know, but you now know enough to start working with git. Here's a simple workflow for safely using git:

`git pull` before starting on a new bit of code, to sync to the remote repo before making any changes
make changes locally, e.g. implementing a new feature; if you have two independent features to implement, complete the first and commit it before moving on to the second
`git diff` to look over the changes you've made before committing them (code review)
`git add` to stage the changes
`git status` to make sure you aren't forgetting to add any files
`git commit` to commit the changes, your message should be descriptive, each commit should represent an ***independent***, ***meaningful***, ***atomic*** unit of work
`git pull` in case there have been changes to the remote repo
correct any merge conflicts
compile, test, and commit any final fixes locally
`git push`

Next Steps

Branching

Lightweight branching is a core, very useful feature of git. It allows you to work on multiple features (feature branches) while maintaining a working version of the code (a “mainline” or “maintenance” branch). It can also let you try different things locally without having them collide.

Read more about a basic feature-branch workflow here:
https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow

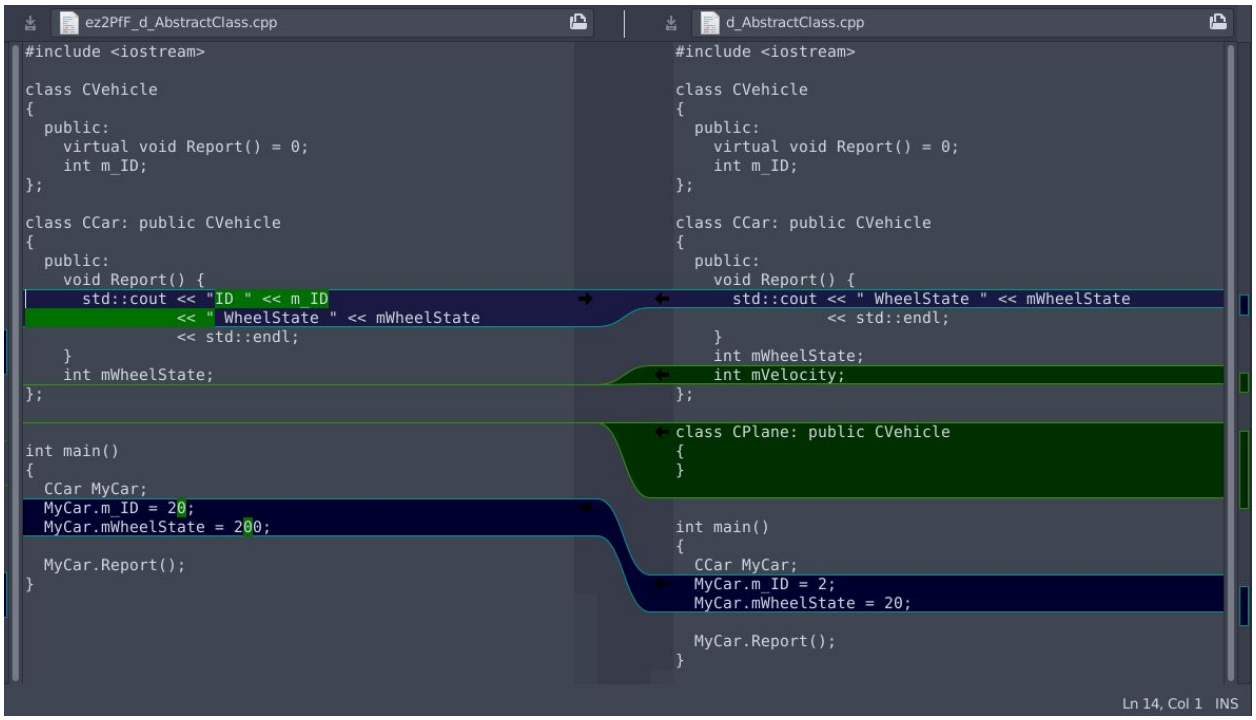
More on using branches: https://www.atlassian.com/git/tutorials/using-branches

And a more complete tutorial is here:
https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

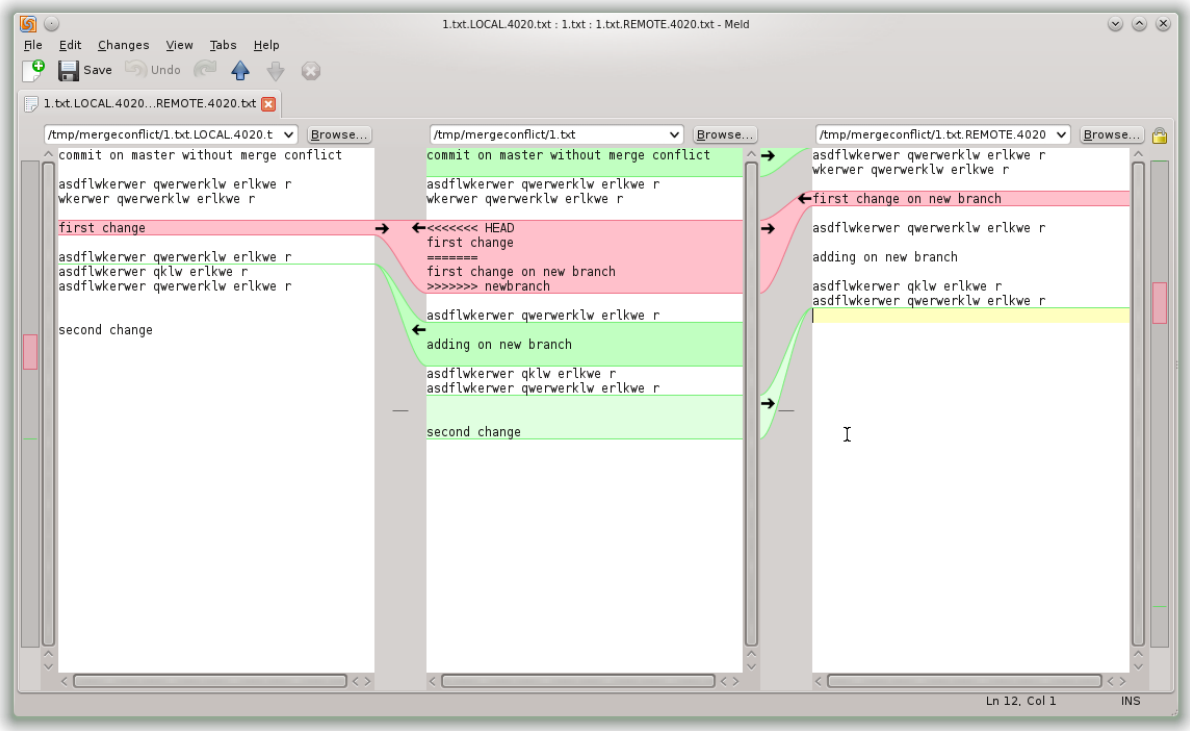
A GUI for Diff and Merging

Understanding differences between files and conducting three-way merges can be greatly facilitated using a graphical “merge” tool like `meld`. Git can be configured to call this automatically when you ask for a diff or merge, e.g. by following these instructions:
https://blog.programster.org/ubuntu-configure-git-to-use-meld-for-diffs

THE UNIVERSITY OF SYDNEY



Meld providing a graphical view of "git diff"



Using meld to perform a 3-way merge to deal with a merge conflict

THE UNIVERSITY OF SYDNEY

Git GUIs

There are a variety of graphical user interfaces that can facilitate interaction with git. gitk provides basic visualisation of the commit history.

Gitkraken is a more powerful GUI, though its licensing may not permit working with private repos: https://www.gitkraken.com/student-resources

Another good option is smartgit https://www.syntevo.com/smartgit/

Many online repo hosts including github and bitbucket also provide graphical means of visualising commit histories and interacting with repos, allowing branching, merging, and editing of code right in the browser. Explore their menu systems thoroughly.

Revision Control for Everything

Git is great for working with code, but you can put other kinds of work under revision control as well. You may find more and more of your work making its way into a repo, as a way of adding change tracking and undo, working on different computers, and facilitating teamwork.

One word of caution is that native git is not a natural fit for revision control of very large assets. For that you can consider extensions to git like git large file storage (LFS): https://git-lfs.github.com/ or an asset management tool designed for the kind of large file you're working with, e.g. for "visual designers" (artists) Alienbrain http://www.alienbrain.com/ has been around for over 20 years and is a mature tool for dealing with heavy graphics and multimedia assets.

THE UNIVERSITY OF SYDNEY

More useful commands

If you can think it, there's probably a git command for it. Try an internet search for something you'd like to get done with git. Some examples:

What you want to do	**Git command**
Get info about the remote git repo	`git remote show origin`
Show changes in already-staged files	`git diff --cached`
Show changes in a previous commit with hash <hash>	`git diff <hash>^!`
Store local changes temporarily	`git stash`
List locally stored changes	`git stash list`
Retrieve locally stored changes	`git stash pop`

Restore to a previous repo version	`git reset --hard <hash>`
	`git push`
	`see also: git reset --soft`

Reset to remote version if on master branch	`git reset --hard origin/master`

Found some more useful git commands? Share them on the MTRX3760 #revision-control channel.